4.2 Angular dla AI Developera: Architektura komponentów i szybka integracja z backendem Node.js

Moduł 4: Szybkie Wdrażanie (Ship Fast) w oparciu o Angular

Krzysztof Rytlewski · 2026

Fundamenty architektury Angular dla aplikacji AI

Angular to opiniowany framework — narzuca określoną strukturę, co jest zaletą przy pracy zespołowej i skalowaniu. Kluczowe elementy i ich role w aplikacji AI:

Element	Rola w aplikacji AI
Component	Jednostka UI — okno czatu, karta wyniku, panel ustawień
Service	Logika biznesowa i komunikacja z API (OpenAI, własny backend)
HttpClient	Wysyłanie zapytań HTTP do Node.js / API zewnętrznych
Router	Nawigacja między widokami (czat, historia, ustawienia)
Signal / RxJS	Reaktywne zarządzanie stanem (odpowiedź AI w czasie rzeczywistym)

W nowoczesnym Angularze (v17+) dominują standalone components — importujące zależności bezpośrednio, bez przynależności do NgModule.

Wzorzec Smart/Dumb Components — separacja odpowiedzialności

Separacja odpowiedzialności jest kluczem do utrzymywalnego kodu. Wzorzec Smart / Dumb (Container / Presentational) naturalnie pasuje do aplikacji AI:

- **Smart component** (kontener) — odpytuje serwis, zarządza stanem, obsługuje błędy. Nie wie, jak renderować dane.

- **Dumb component** (prezentacyjny) — przyjmuje dane przez `@Input()`, emituje zdarzenia przez `@Output()`. Nie wie, skąd dane pochodzą.

Podział sprawia, że komponenty prezentacyjne są w pełni wielokrotnego użytku — ta sama lista wiadomości może działać zarówno w widoku czatu, jak i w podglądzie historii.

Smart component — ChatContainerComponent

Smart component wstrzykuje serwis, odczytuje sygnały ze stanu i deleguje renderowanie do komponentów dumb.

```
@Component({
  standalone: true,
  imports: [ChatMessagesComponent, ChatInputComponent],
  template: `
    <app-chat-messages [messages]="messages()" />
    <app-chat-input (send)="onSend($event)" [loading]="loading()" />
  `
})
export class ChatContainerComponent {
  private chatService = inject(ChatService);
  messages = this.chatService.messages;
  loading  = this.chatService.loading;
  onSend(text: string) { this.chatService.sendMessage(text); }
}
```

Sygnały (`messages()`, `loading()`) automatycznie odświeżają widok przy każdej zmianie stanu.

Dumb component — ChatInputComponent

Komponent prezentacyjny przyjmuje dane wejściowe i emituje zdarzenia — nie zna źródła danych ani logiki biznesowej.

```
@Component({
  standalone: true,
  imports: [MatFormFieldModule, MatInputModule, MatButtonModule, FormsModule],
  template: `
    <mat-form-field appearance="outline">
      <textarea matInput [(ngModel)]="text"></textarea>
    </mat-form-field>
    <button mat-raised-button color="primary"
            [disabled]="loading || !text.trim()"
            (click)="send.emit(text); text = ''">Wyślij</button>
  `
})
export class ChatInputComponent {
  @Input() loading = false;
  @Output() send = new EventEmitter<string>();
  text = '';
}
```

Serwisy Angular i komunikacja z backendem Node.js

Serwis (`@Injectable`) to singleton zarządzający logiką i stanem aplikacji. Serwis czatu wysyła wiadomości do backendu Node.js i aktualizuje sygnały stanu.

Backend Node.js/Express udostępnia endpoint `POST /api/chat`, który przyjmuje `{ message, history }`, wywołuje OpenAI API i zwraca `{ reply }`. Serwis Angular wysyła żądanie przez `HttpClient` i po otrzymaniu odpowiedzi dołącza wiadomość asystenta do tablicy sygnału `messages`.

Sygnał `loading` pozwala blokować interfejs podczas oczekiwania na odpowiedź AI — przycisk wysyłania jest wyłączony, gdy `loading()` zwraca `true`.

Proxy deweloperski Angular CLI — rozwiązanie problemu CORS

Podczas developmentu Angular działa na `localhost:4200`, a backend Node.js na `localhost:3000`. Przeglądarki blokują takie żądania cross-origin (CORS). Angular CLI oferuje wbudowany **proxy**:

```
// proxy.conf.json
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true
  }
}
```

Następnie w `angular.json` w sekcji `serve.options` dodajemy `"proxyConfig": "proxy.conf.json"`. Żądania do `/api/*` z Angulara są transparentnie przekierowywane do Node.js bez błędów CORS.

Reaktywność z Angular Signals

Angular Signals (stabilne od v17) to prostszy model reaktywności niż RxJS dla podstawowych przypadków. Aktualizacja sygnału automatycznie odświeża powiązane widoki.

Sygnał tworzymy przez `signal(wartość)`, odczytujemy go jak funkcję — `messages()`, aktualizujemy przez `.set()` lub `.update()`:

```
messages = signal<Message[]>([]);
loading  = signal(false);

// Dodaj nową wiadomość do istniejącej tablicy:
this.messages.update(msgs => [...msgs, newMsg]);
```

Dla prostych przypadków (stan listy, flaga ładowania) Signals są czytelniejsze i wymagają mniej kodu niż `BehaviorSubject` z RxJS.

Strumieniowanie odpowiedzi AI — Fetch API z ReadableStream

Aby wyświetlać odpowiedź AI słowo po słowie (jak w ChatGPT), stosujemy `fetch` z `ReadableStream` zamiast standardowego `HttpClient`:

Asynchroniczna metoda `streamMessage()` w serwisie Angular dodaje pustą wiadomość asystenta do tablicy, a następnie w pętli odczytuje kolejne fragmenty (chunki) z odpowiedzi HTTP i dokłada je do ostatniej wiadomości przez `signal.update()`.

Widok automatycznie aktualizuje się z każdym fragmentem — użytkownik widzi przyrastający tekst w czasie rzeczywistym. Backend musi obsługiwać endpoint strumieniujący dane (np. `POST /api/chat/stream` z OpenAI streaming API).

Konfiguracja środowisk — environment.ts

Angular CLI obsługuje pliki środowiskowe (`environments/`), które pozwalają definiować różne wartości dla środowisk deweloperskiego i produkcyjnego:

```
// environments/environment.ts (dev)
export const environment = {
  production: false,
  apiUrl: '/api',   // proxy deweloperski
};

// environments/environment.prod.ts (prod)
export const environment = {
  production: true,
  apiUrl: 'https://xyz.execute-api.eu-west-1.amazonaws.com/prod',
};
```

Dzięki temu ten sam kod serwisu Angular działa zarówno lokalnie (z proxy na Node.js w Dockerze), jak i w produkcji (z AWS Lambda przez API Gateway) — bez żadnych zmian w kodzie aplikacji.

Podsumowanie architektury Angular dla AI developera

Kompletna architektura aplikacji AI w Angular opiera się na czterech filarach:

1. **Standalone components** z wzorcem Smart/Dumb — czytelny podział odpowiedzialności między zarządzanie stanem a prezentację.

2. **Serwisy z Signals** — reaktywny stan współdzielony między komponentami, prosty i czytelny.

3. **Proxy deweloperski + environments** — spójne API URL w dev i prod, brak problemów z CORS.

4. **Fetch API / HttpClient** — elastyczna komunikacja z backendem, obsługa zarówno zwykłych żądań, jak i strumieniowania.

Docker Compose spina całość: Angular dev server, Node.js i MongoDB działają razem jedną komendą.